

Лабораторная работа №24. Современный JavaScript

Цель работы:

- Изучить и закрепить ключевые возможности современного JavaScript (ES6+);
- Освоить деструктуризацию, spread/rest операторы, модули;
- Научиться работать с асинхронным кодом (промисы).

Краткая теория (вводная часть)

Современный JavaScript (ES6+) — это набор возможностей языка, появившихся начиная с версии ECMAScript 2015 (ES6) и продолжающих развиваться.

Основные нововведения ES6+:

- **деструктуризация** — удобное извлечение данных из объектов и массивов;
- **spread/rest операторы** — работа с коллекциями данных;
- **модули** — разделение кода на файлы;
- **промисы и async/await** — работа с асинхронным кодом;
- **fetch API** — запросы к серверу;
- **классы** — более привычный синтаксис ООП;
- **template literals** — удобная работа со строками.

Зачем это нужно?

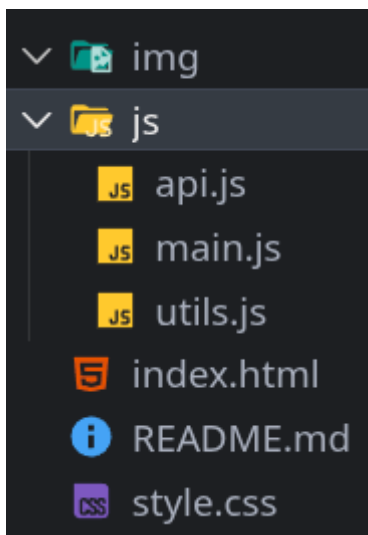
- **React, Vue, Angular** — все современные фреймворки используют ES6+;
- **Чистый код** — современный синтаксис делает код читаемым;
- **Асинхронность** — веб-приложения активно работают с API;
- **Модульность** — большие проекты разбиваются на модули.

Шаг 1. Подготовка рабочего окружения

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:

Lab24_ModernJS

5. Перейдите в созданную директорию
6. Создайте структуру проекта:



7. Откройте созданную папку в **Visual Studio Code**

Шаг 2. Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:

git init

2. Создайте файл .gitignore:

node_modules/

.DS_Store

***.log**

3. Добавьте файлы в индекс:

git add .

4. Создайте первый коммит:

git commit -m "Initial commit"

Шаг 3. Создание удалённого репозитория

1. Создайте **пустой public-репозиторий** на GitHub с именем:

Lab24_ModernJS_FIO

2. Не добавляйте:

- **README**
- **.gitignore**
- **License**

3. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

4. Отправьте файлы:

git push -u origin main

5. Сделайте **скриншот терминала** с выполненными командами и сохраните в папку **img** под именем:

gitPushLab24_FIO.png

6. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot"

git push

Шаг 4. Базовая HTML-структура

1. Откройте файл **index.html** и создайте базовую разметку (используйте **!** + **Enter**):

```
<!DOCTYPE html>
<html lang="ru-Ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Lab24-26 - Modern JavaScript</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body></body>
</html>
```

2. Добавьте контейнер:

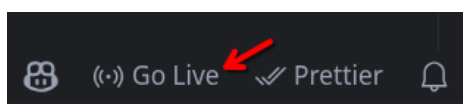
.container>h1+h2^script

3. Внесите дополнительные правки:

```
<div class="container">
  <h1>Lab 24-26 - Modern JavaScript</h1>
  <h2>Современный JavaScript</h2>
</div>
<script type="module" src="js/api.js"></script>
```

Важно: `type="module"` — это способ подключения JS-файлов, который позволяет использовать **import/export**.

4. Запустите вашу страницу через **Live Server**, чтобы наблюдать изменения:



5. Сейчас у нас отображается страница в следующем виде:

Lab 24-26 - Modern JavaScript

Современный JavaScript

Шаг 5. Базовые CSS-стили

6. Откройте **styles.css** и добавьте базовые стили:

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Для всех элементов убрали отступы и внутренние поля, а также изменили модель расчёта размеров на border-box:

Lab 24-26 - Modern JavaScript

Современный JavaScript

Далее вносим изменения в тело страницы:

```
body {  
  font-family: Verdana, sans-serif;  
  background: #667eea;  
  min-height: 100vh;  
  padding: 20px;  
}
```

Поменяли шрифт, фоновый цвет, установили минимальную высоту на весь экран и добавили внутренние отступы:

Lab 24-26 - Modern JavaScript

Современный JavaScript

Далее для контейнера:

```
.container {  
  max-width: 900px;  
  margin: 0 auto;  
  background: white;  
  padding: 30px;  
  border-radius: 10px;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
}
```

Для контейнера ограничили ширину, выровняли по центру, задали белый фон, внутренние отступы и скруглённые углы:

Lab 24-26 - Modern JavaScript

Современный JavaScript

Для заголовка **h1** изменим цвет и уменьшили нижний отступ:

```
h1 {  
  color: #333;  
  margin-bottom: 10px;  
}
```

Для заголовка **h2** изменим цвет, уменьшили жирность шрифта и увеличили нижний отступ:

```
h2 {  
  color: #667eea;  
  margin-bottom: 30px;  
  font-weight: 400;  
}
```

Итог:

Lab 24-26 - Modern JavaScript

Современный JavaScript

Практическое задание

1. Сделайте скриншот **страницы в браузере** и сохраните в папку **img** с именем: **step5_htmlCss_FIO.png**

2. Выполните следующие команды:

git add .

git commit -m "Added Html and CSS structure"

git push

Шаг 6. Деструктуризация

6.1. Теоретическая часть

Деструктуризация — способ извлечения данных из массивов и объектов в отдельные переменные.

Зачем это нужно?

- **Меньше кода** — не нужно писать `obj.prop` много раз;
- **Читаемость** — сразу видно, какие данные используются;
- **React** — деструктуризация используется повсеместно.

6.2. Деструктуризация массивов

Откройте `js/main.js`. Добавим вывод заголовка раздела в консоль:

```
console.log("\n=== Деструктуризация массивов ===");
```

Создадим массив `colors` из трёх цветов:

```
const colors = ["red", "green", "blue"];
```

Старый способ:

Получим отдельные элементы массива через индексы `colors[0]` и `colors[1]`, сохраним их в переменные `color1` и `color2`, затем выведем на консоль:

```
const color1 = colors[0];  
const color2 = colors[1];  
console.log(color1, color2);
```

Новый способ:

Используем деструктуризацию для одновременного извлечения всех элементов массива в отдельные переменные `firstColor`, `secondColor`, `thirdColor`:

```
const [firstColor, secondColor, thirdColor] = colors;  
console.log(firstColor, secondColor, thirdColor);
```

Пропуск элементов:

Если нужны только первый и третий элементы, оставим пустое место вместо второго элемента (используя двойную запятую):

```
const [primary, , tertiary] = colors;  
console.log(primary, tertiary);
```

Значения по умолчанию:

Можем задать значение по умолчанию для элемента, которого нет в массиве. Если четвёртого элемента нет, переменная **c4** получит значение **"yellow"**:

```
const [c1, c2, c3, c4 = "yellow"] = colors;  
console.log(c4); // yellow
```

Запустите программу через терминал:

```
➤$ node js/main.js  
  
=== Деструктуризация массивов ===  
red green  
red green blue  
red blue  
yellow
```

Чтобы вывод в терминал оставался компактным, рекомендуется комментировать в коде все успешно выполненные шаги — это поможет избежать перегрузки консольного вывода.

6.3. Деструктуризация объектов

Добавим вывод заголовка раздела в **main.js** :

```
console.log("Деструктуризация объектов");
```

Создадим объект **user** с информацией о пользователе (имя, возраст, город):

```
const user = {  
  name: "Алиса",  
  age: 25,  
  city: "Москва",  
};
```

Старый способ:

Получим значения свойств объекта через точечную нотацию **user.name** и **user.age**, сохраним в переменные:

```
const userName = user.name;  
const userAge = user.age;  
console.log(userName, userAge);
```

Новый способ:

Используем деструктуризацию для извлечения свойств объекта. Имена переменных должны совпадать с именами свойств:

```
const { name, age, city } = user;  
console.log(name, age, city);
```

Переименование при деструктуризации:

Если нужны другие имена переменных, используем синтаксис **свойство: новоеИмя**:

```
const { name: fullName, age: years } = user;  
console.log(fullName, years);
```

Значения по умолчанию:

Задаём значение по умолчанию для свойства, которого нет в объекте (например, **country**):

```
const { name: personName, country = "Россия" } = user;  
console.log(personName, country);
```

Запустите программу через терминал:

```
<->$ node js/main.js  
Деструктуризация объектов  
Алиса 25  
Алиса 25 Москва  
Алиса 25  
Алиса Россия
```

Закомментируйте выше написанный код.

6.4. Деструктуризация в параметрах функций

Добавим вывод заголовка раздела в **main.js** :

```
console.log("Деструктуризация в параметрах");
```

Раскомментируйте код с объектом user:

```
const user = {  
  name: "Алиса",  
  age: 25,  
  city: "Москва",  
};
```

Старый способ:

Создадим функцию, которая принимает объект целиком, затем внутри функции обращаемся к каждому свойству через точечную нотацию:

```
function printUserOld(user) {  
  console.log(`Имя: ${user.name}`);  
  console.log(`Возраст: ${user.age}`);  
  console.log(`Город: ${user.city}`);  
}
```

Новый способ:

Используем деструктуризацию прямо в параметрах функции. Это сразу извлекает нужные свойства:

```
function printUser({ name, age, city }) {  
  console.log(`Имя: ${name}`);  
  console.log(`Возраст: ${age}`);  
  console.log(`Город: ${city}`);  
}
```

Вызовем функции, передав объект **user**:

```
printUserOld(user);  
console.log();  
printUser(user);
```

Запустите программу через терминал, чтобы увидеть одинаковый результат работы:

```
➤$ node js/main.js  
Деструктуризация в параметрах  
Имя: Алиса  
Возраст: 25  
Город: Москва  
  
Имя: Алиса  
Возраст: 25  
Город: Москва
```

Закомментируйте выше написанный код.

6.5. Практическое задание

Задание:

1. Создайте объект **product** со свойствами: **name**, **price**, **category**, **inStock**
2. Используя деструктуризацию, извлеките все свойства

3. Создайте функцию **printProduct**, которая принимает объект и использует деструктуризацию в параметрах

4. Выведите информацию о продукте

Практическое задание

1. Сделайте скриншот/ы файла **main.js** и сохраните в папку **img** с именем:

step6_Js_destructuring_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Destructuring in JS has been studied"

git push

Шаг 7. Spread и Rest операторы

7.1. Теоретическая часть

Spread operator (...) — разворачивает массивы/объекты.

Rest operator (...) — собирает элементы в массив.

Синтаксис одинаковый, но применение разное!

7.2. Spread оператор для массивов

В **main.js** добавим вывод заголовка раздела:

```
console.log("Spread для массивов");
```

Создадим два массива чисел **arr1** и **arr2**:

```
const arr1 = [1, 2, 3];  
const arr2 = [4, 5, 6];
```

Объединение массивов:

Используем оператор **spread (...)** для объединения двух массивов в один. **Spread** "разворачивает" элементы массива:

```
const combined = [...arr1, ...arr2];  
console.log("Объединённый массив:", combined);
```

Копирование массива:

Создадим копию массива **arr1**. Это важно, чтобы изменения в копии не влияли на оригинал:

```
const copy = [...arr1];
console.log("Копия массива:", copy);
```

Добавление элементов:

Добавим новые элементы в начало и конец массива при копировании:

```
const extended = [0, ...arr1, 7, 8];
console.log("Расширенный массив:", extended);
```

Запустите программу через **терминал**:

```
➤$ node js/main.js
Spread для массивов
Объединённый массив: [ 1, 2, 3, 4, 5, 6 ]
Копия массива: [ 1, 2, 3 ]
Расширенный массив: [ 0, 1, 2, 3, 7, 8 ]
```

Закомментируйте выше написанный код.

7.3. Spread оператор для объектов

Добавим вывод заголовка раздела:

```
console.log("Spread для объектов");
```

Создадим два объекта: **person** с базовой информацией и **address** с адресом:

```
const person = {
  name: "Иван",
  age: 30,
};
```

```
const address = {
  city: "Санкт-Петербург",
  street: "Невский проспект",
};
```

Объединение объектов:

Используем `spread` для слияния двух объектов в один. Все свойства обоих объектов будут в **fullInfo**:

```
const fullInfo = { ...person, ...address };
console.log("Полная информация:", fullInfo);
```

Копирование объекта:

Создадим независимую копию объекта **person**:

```
const personCopy = { ...person };
console.log("Копия объекта:", personCopy);
```

Переопределение свойств:

При объединении объектов, если есть одинаковые ключи, последнее значение перезаписывает предыдущее:

```
const updated = { ...person, age: 31, occupation: 'Developer' };
console.log("Обновлённый объект:", updated);
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Spread для объектов
Полная информация: {
  name: 'Иван',
  age: 30,
  city: 'Санкт-Петербург',
  street: 'Невский проспект'
}
Копия объекта: { name: 'Иван', age: 30 }
Обновлённый объект: { name: 'Иван', age: 31, occupation: 'Developer' }
```

Закомментируйте выше написанный код.

7.4. Rest оператор

Добавим вывод заголовка раздела:

```
console.log("Rest оператор");
```

В параметрах функции:

Создадим функцию **sum**, которая может принимать любое количество чисел.

Rest оператор (**...numbers**) собирает все аргументы в массив:

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}
```

Вызовем функцию с разным количеством аргументов:

```
console.log("Сумма 1,2,3:", sum(1, 2, 3));
console.log("Сумма 1,2,3,4,5:", sum(1, 2, 3, 4, 5));
```

Деструктуризация с rest:

Извлечём первый элемент массива в переменную **first**, а все остальные соберём в массив **rest**:

```
const numbers = [10, 20, 30, 40, 50];
const [first, second, ...rest] = numbers;
console.log("Первое число:", first);
console.log("Второе число:", second);
console.log("Остальные числа:", rest);
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Rest оператор
Сумма 1,2,3: 6
Сумма 1,2,3,4,5: 15
Первое число: 10
Второе число: 20
Остальные числа: [ 30, 40, 50 ]
```

Закомментируйте выше написанный код.

7.5. Практическое задание

Задание:

1. Создайте два массива чисел
2. Объедините их с помощью **spread**
3. Создайте функцию **findMax**, которая принимает любое количество чисел (rest) и возвращает максимальное
4. Создайте два объекта и объедините их

Практическое задание

1. Сделайте скриншот/ы файла **main.js** и сохраните в папку **img** с именем: **step7_spreadRestLab24_FIO.png**

2. Выполните следующие команды:

git add .

git commit -m "spread/rest in JS"

git push

Шаг 8. Модули (Import/Export)

8.1. Теоретическая часть

Модули — способ разделения кода на отдельные файлы.

Преимущества:

- **Организация кода** — каждая часть в своём файле;

- **Переиспользование** — функции можно импортировать в разные файлы;
- **Изоляция** — переменные не конфликтуют между файлами;
- **Современный подход** — так работают все фреймворки.

Сравнение с C#:

C#	JavaScript
using System;	import { func } from './file.js'
namespace	модули ES6
public class	export

8.2. Создание модуля utils.js

Откройте `js/utils.js`.

Создадим несколько полезных функций, которые будем экспортировать для использования в других файлах.

Функция приветствия:

Создадим функцию `greet`, которая принимает имя и возвращает приветствие:

```
export function greet(name) {
  return `Привет, ${name}!`;
}
```

Функция сложения:

Создадим функцию `add` для сложения двух чисел:

```
export function add(a, b) {
  return a + b;
}
```

Функция умножения:

Создадим функцию `multiply` для умножения двух чисел:

```
export function multiply(a, b) {
  return a * b;
}
```

Константа PI:

Экспортируем математическую константу:

```
export const PI = 3.14159;
```

Default export:

Создадим главную функцию модуля, которая будет экспортирована по умолчанию:

```
export default function sayHello() {  
  return "Привет из модуля utils!";  
}
```

8.3. Импорт модулей

В файле `js/main.js` добавьте:

Импорт именованных экспортов:

Импортируем конкретные функции и константы из модуля `utils.js` по их именам:

```
import { greet, add, PI } from "./utils.js";
```

Теперь используем импортированные функции:

```
console.log("Модули");  
console.log(greet("Алексей"));  
console.log("5 + 3 =", add(5, 3));  
console.log("Значение PI:", PI);
```

Импорт с переименованием:

Если нужно переименовать импортируемую функцию, используем `as`:

```
import { multiply as умножить } from "./utils.js";  
console.log("4 * 7 =", умножить(4, 7));
```

Импорт всего модуля:

Импортируем все экспорты модуля в один объект `Utils`:

```
import * as Utils from "./utils.js";  
console.log(Utils.greet("Мария"));  
console.log("Умножение:", Utils.multiply(3, 9));
```

Запустите программу через терминал:

```
• L>$ node js/main.js
Модули
Привет, Алексей!
5 + 3 = 8
Значение PI: 3.14159
4 * 7 = 28
Привет, Мария!
Умножение: 27
```

Закомментируйте выше написанный код в **main.js**.

8.5. Практическое задание

Задание:

1. Создайте файл **js/math.js**
2. Создайте, продумайте логику и экспортируйте функции: **square** и **cube**
3. Создайте и экспортируйте константу **E = 2.718**
4. Импортируйте всё это в **main.js**
5. Протестируйте функции

Практическое задание

1. Сделайте скриншот файла **math.js** и сохраните в папку **img** с именем:
step8_modulesLab24_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "modules in JS"

git push

Шаг 9. Промисы (Promises)

9.1. Теоретическая часть

Promise — объект, представляющий результат асинхронной операции.

Асинхронность означает, что код выполняется **не последовательно**, а **параллельно**.

Состояния промиса:

- **pending** — ожидание (начальное состояние)
- **fulfilled** — выполнено успешно

- **rejected** — выполнено с ошибкой

Зачем нужны промисы?

- **Запросы к серверу** — данные приходят не сразу
- **Чтение файлов** — операция занимает время
- **Таймеры** — задержка выполнения
- **Избежание callback hell** — вложенных функций обратного вызова

Сравнение с C#:

C#	JavaScript
async Task<T>	Promise<T>
await	await или .then()
try/catch	.catch() или try/catch

9.2. Создание промиса

В **main.js** добавьте.

Добавим вывод заголовка раздела:

```
console.log("Промисы");
```

Создание простого промиса:

Создадим промис, который имитирует выполнение операции. В зависимости от значения переменной **success** промис либо успешно завершится (**resolve**), либо вернёт ошибку (**reject**):

```
const simplePromise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve("Операция выполнена успешно!");  
  } else {  
    reject("Произошла ошибка!");  
  }  
});
```

Использование промиса:

Обработаем результат промиса: **.then()** срабатывает при успехе, **.catch()** — при ошибке:

```
simplePromise  
  .then((result) => console.log("Результат:", result))  
  .catch((error) => console.log("Ошибка:", error));
```

Промис с задержкой:

Создадим функцию **delay**, которая возвращает промис, завершающийся через заданное время (имитация асинхронной операции):

```
function delay(ms) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(`Прошло ${ms} миллисекунд`);  
    }, ms);  
  });  
}
```

Вызовем функцию с задержкой 1 секунда:

```
delay(1000)  
  .then((message) => console.log(message));
```

Имитация загрузки данных:

Создадим функцию **fetchUserData**, которая имитирует запрос к серверу. Промис завершится успешно через 1.5 секунды, если **userId** больше 0:

```
function fetchUserData(userId) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      if (userId > 0) {  
        resolve({  
          id: userId,  
          name: "Иван Иванов",  
          email: "ivan@example.com",  
        });  
      } else {  
        reject("Неверный ID пользователя");  
      }  
    }, 1500);  
  });  
}
```

Вызовем функцию и обработаем результат:

```
fetchUserData(1)  
  .then((user) => console.log("Пользователь:", user))  
  .catch((error) => console.log("Ошибка:", error));
```

Цепочка промисов:

Создадим три функции, которые выполняются последовательно, каждая передаёт результат следующей:

```
function step1() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Шаг 1 завершён"), 500);  
  });  
}
```

```
function step2(previousResult) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(`${previousResult} -> Шаг 2 завершён`), 500);  
  });  
}
```

```
function step3(previousResult) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(`${previousResult} -> Шаг 3 завершён`), 500);  
  });  
}
```

Выполним цепочку: каждый `.then()` ждёт завершения предыдущего промиса и передаёт результат дальше:

```
step1()  
  .then((result1) => step2(result1))  
  .then((result2) => step3(result2))  
  .then((finalResult) => console.log("Финальный результат:", finalResult))  
  .catch((error) => console.log("Ошибка в цепочке:", error));
```

Запустите программу через **терминал**:

```
➤$ node js/main.js  
Промисы  
Результат: Операция выполнена успешно!  
Прошло 1000 миллисекунд  
Пользователь: { id: 1, name: 'Иван Иванов', email: 'ivan@example.com' }  
Финальный результат: Шаг 1 завершён -> Шаг 2 завершён -> Шаг 3 завершён
```

Закомментируйте выше написанный код в `main.js`.

9.3. Практическое задание

Задание:

1. Создайте функцию **checkInventory**, которая возвращает промис
2. Если товар в наличии — **resolve** с сообщением
3. Если товара нет — **reject** с ошибкой
4. Используйте **.then()** и **.catch()** для обработки результата

Практическое задание

1. Сделайте скриншот/ы файла **main.js** и сохраните в папку **img** с именем:
step9_promisesLab24_FIO.png
2. Выполните следующие команды:

git add .

git commit -m "promises in JS"

git push